

Part. 1

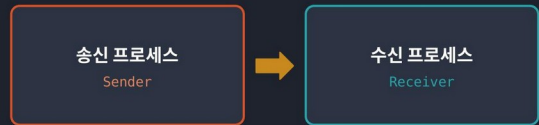
IPC

Inter-Process Communication

프로세스 간 통신

발표자 Janghwan Kim (Harley)

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증



- 01 전역변수 + 뮤텍스 / 세마포어
- 02 공유메모리 + 링버퍼 (메시지 큐)
- 03 TCP / IP 소켓

발표 대본

표지 · 약 30 초

안녕하십니까 . Part.1 과제인 IPC, 프로세스 간 통신 발표를 맡은 Janghwan Kim 입니다 .

이번 과제의 목표는 " 멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능을 구현하고 검증하는 것 " 입니다 .

발표는 크게 두 덩어리입니다 . 앞부분에서는 프로세스와 쓰레드가 무엇이고 어떤 방법으로 데이터를 주고받는지 , 그리고 IPC 수단들의 특징과 장단점을 정리하겠습니다 . 뒷부분에서는 그 개념들이 실제 코드에서 어떻게 쓰였는지 ToDo 세 개를 실행 결과와 함께 보여드리겠습니다 .

목표

멀티쓰레드 · 멀티프로세스 환경에서의 데이터 송수신 기능 구현 및 검증

01

쓰레드 / 프로세스 개념 및 통신 방법

개념

리눅스 환경에서 프로그램 · 프로세스 · 쓰레드가 무엇이고, 서로 어떻게 데이터를 주고받는지 정리한다.

02

IPC 개념 및 뮤텍스 / 세마포어

개념

UDP · TCP/IP · MessageQ · 링버퍼 · 공유메모리의 특징과 사용 상황, 장 / 단점. 그리고 이를 안전하게 묶어주는 동기화 수단.

03

데이터 송 / 수신 기능 구현 및 코드 정리

구현

송신 · 수신 쓰레드를 구성해 1 부터 100 까지 0.1 초 간격으로 주고받는다.
ToDo#1 쓰레드 간 · ToDo#2 프로세스 간 · ToDo#3 TCP / IP

오늘 발표는 1 → 2 → 3 순서로, 개념을 먼저 정리하고 그 개념이 코드 어디에 쓰였는지 이어서 보여드립니다.

발표 대본

목표 · 약 1 분

먼저 오늘 다룰 세 가지입니다.

첫 번째, 리눅스 환경에서의 쓰레드와 프로세스 개념, 그리고 통신 방법입니다. 프로그램 · 프로세스 · 쓰레드가 무엇이 다른지, 그 차이 때문에 통신 방법이 왜 갈리는지를 정리합니다.

두 번째, IPC 개념과 동기화입니다. UDP, TCP/IP, 메시지 큐, 링버퍼, 공유메모리를 각각 특징 · 어떤 상황에서 쓰는지 · 장단점으로 나눠 보고, 이들을 안전하게 묶어 주는 뮤텍스와 세마포어를 다룹니다.

세 번째, 실제 구현입니다. 송신 쓰레드와 수신 쓰레드를 구성해서 1 부터 100 까지 0.1 초 간격으로 주고받는 기능을 ToDo 세 가지 방식으로 구현했고, 실행 결과까지 보여드리겠습니다.

순서는 개념을 먼저 정리하고, 그 개념이 코드 어디에 쓰였는지 이어서 보여드리겠습니다.

01

쓰레드 / 프로세스 개념 및 통신 방법

리눅스 환경 기준

1-1

프로그램 · 프로세스 · 쓰레드

무엇이 다르고, 왜 그 차이 때문에 통신 방법이 갈리는가

1-2

통신 방법

쓰레드 간 · 프로세스 간에 쓸 수 있는 수단과 선택 기준

발표 대본

섹션 1 · 약 20 초

첫 번째 주제입니다. 리눅스 환경 기준으로 쓰레드와 프로세스 개념, 그리고 통신 방법을 보겠습니다.

두 가지로 나눠서, 먼저 1-1 에서 프로그램 · 프로세스 · 쓰레드가 무엇이 다른지 정리하고, 1-2 에서 그 차이에 따라 쓸 수 있는 통신 방법이 어떻게 갈리는지 보겠습니다.

1-1 프로그램 · 프로세스 · 쓰레드

무엇을 공유하고 무엇을 따로 갖는지가 통신 방법을 결정한다

1 프로그램 Program

디스크에 저장된 실행 파일 . 아직 실행되지 않은 명령어와 데이터의 묶음이다 .

• 정적 — 메모리도 자원도 할당되지 않은 상태

2 프로세스 Process

실행 중인 프로그램 . 커널이 독립된 가상 주소공간과 자원 (PID, 파일 디스크립터) 을 부여준 단위다 .

• 서로의 메모리를 볼 수 없다 → 그래서 IPC 가 필요하다

3 쓰레드 Thread

프로세스 안의 실행 흐름 . Code · Data · Heap 은 공유하고 Stack 과 레지스터만 따로 갖는다 .

• 전역변수로 바로 주고받는다 → 대신 동기화가 필수다

리눅스에서는 프로세스도 쓰레드도 커널 입장에서 task_struct 하나 . clone() 에 무엇을 공유할지만 다르게 준다 .

프로세스 A (독립된 가상 주소공간)

쓰레드끼리 공유하는 영역

Code

Data (전역변수)

Heap

파일 디스크립터

쓰레드 1 (송신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

쓰레드 2 (수신)

Stack · 레지스터 (PC · SP)

쓰레드마다 따로

전역변수를 통해 복사 없이 바로 전달 · 단, 동시 접근은 뮤텝스로 보호



프로세스 B

주소공간이 다르다 → 직접 접근 불가

발표 대본

1-1 프로그램 · 프로세스 · 쓰레드 · 약 2 분

세 가지를 한 줄씩 구분하면 이렇습니다 .

프로그램은 디스크에 저장된 실행 파일입니다 . 아직 실행되지 않은 명령어와 데이터의 묶음이고 , 메모리도 자원도 할당되지 않은 정적인 상태입니다 .

프로세스는 그 프로그램이 실행된 것입니다 . 커널이 독립된 가상 주소공간과 PID, 파일 디스크립터 같은 자원을 부여준 단위입니다 . 여기서 중요한 건 , 프로세스끼리는 서로의 메모리를 볼 수 없다는 점입니다 . 바로 이것 때문에 IPC 라는 별도의 수단이 필요합니다 .

쓰레드는 프로세스 안의 실행 흐름입니다 . 오른쪽 그림을 보시면 , Code · Data · Heap · 파일 디스크립터는 쓰레드끼리 공유하고 , 스택과 레지스터만 쓰레드마다 따로 갖습니다 . 그래서 쓰레드끼리는 전역변수 하나로 바로 데이터를 넘길 수 있습니다 . 대신 두 쓰레드가 같은 변수를 동시에 건드리면 값이 깨지기 때문에 동기화가 반드시 필요합니다 .

한 가지 덧붙이면 , 리눅스에서는 프로세스와 쓰레드가 커널 입장에서 사실 같은 구조체입니다 . 둘 다 task_struct 하나이고 , clone() 을 호출할 때 주소공간을 공유할지 말지만 다르게 주는 것뿐입니다 . 그래서 리눅스에서는 쓰레드 생성 비용이 프로세스보다 훨씬 가볍습니다 .

1-2 통신 방법

주소공간을 공유하느냐 아니냐에 따라 쓸 수 있는 수단이 완전히 갈린다

쓰레드 간 통신

같은 주소공간 · 커널을 거치지 않는다

전역변수 · 공유 자료구조

+ 뮤텝스 · 세마포어 · 조건변수

메모리 주소를 그대로 넘긴다 (복사 0 회)

- 가장 빠르다 — 커널 개입도 복사도 없다
- 동기화를 빠뜨리면 데이터가 깨지거나 데드락
- 한 쓰레드가 죽으면 프로세스 전체가 같이 죽는다

ToDo#1 이 방식으로 구현

프로세스 간 통신 (IPC)

주소공간이 분리 → 반드시 커널이 만든 통로를 거친다

파이프 · FIFO	단방향 바이트 스트림 (부모-자식 · 이름)	pipe() · mkfifo()
메시지 큐	메시지 단위로 넣고 뺌다 · 경계 보존	msgsnd / msgrcv
공유메모리	같은 물리 메모리를 양쪽에 매핑 (복사 0 회)	shmget · mmap
시그널	데이터 없이 이벤트 발생만 알린다	kill() · signal()
소켓	유일하게 다른 장비까지 확장된다	socket / connect

선택 기준

같은 프로세스 안인가 ?

전역변수 + 뮤텝스 / 세마포어

ToDo#1

같은 PC 의 다른 프로세스인가 ?

메시지 큐 · 공유메모리

ToDo#2

다른 PC · 다른 프로그램인가 ?

소켓 (TCP / UDP)

ToDo#3

발표 대본

1-2 통신 방법 · 약 2 분

통신 방법은 " 주소공간을 공유하느냐 " 하나로 갈립니다 .

왼쪽 , 쓰레드 간 통신입니다 . 같은 주소공간이니까 전역변수나 공유 자료구조를 그대로 쓰면 됩니다 . 메모리 주소를 넘기는 것이라 복사도 없고 커널도 거치지 않아서 가장 빠릅니다 . 대신 동기화를 빠뜨리면 데이터가 깨지거나 데드락이 나고 , 한 쓰레드가 죽으면 프로세스 전체가 같이 죽습니다 . ToDo#1 이 이 방식입니다 .

오른쪽 , 프로세스 간 통신입니다 . 주소공간이 분리돼 있으니 반드시 커널이 만들어 준 통로를 거쳐야 합니다 . 파이프는 단방향 바이트 스트림이고 , 메시지 큐는 메시지 단위로 넣고 빼며 커널이 경계와 순서를 지켜 줍니다 . 공유메모리는 같은 물리 메모리를 양쪽에 매핑해서 복사가 없어 가장 빠릅니다 . 시그널은 데이터 없이 이벤트만 알리고 , 소켓은 이 중에서 유일하게 다른 장비까지 확장됩니다 .

아래 선택 기준이 오늘 과제 구조와 그대로 맞습니다 . 같은 프로세스 안이면 전역변수에 뮤텝스와 세마포어를 붙이고 , 같은 PC 의 다른 프로세스면 메시지 큐나 공유메모리를 쓰고 , 다른 PC 나 다른 프로그램이면 소켓을 씁니다 . 각각 ToDo#1, #2, #3 입니다 .

02

IPC 개념 및 뮤텍스 / 세마포어

각 수단의 특징 · 어떤 상황에서 쓰는지 · 장 / 단점

2-1

UDP · TCP / IP

네트워크를 건너는 통신 · 신뢰성이나 속도나

2-2

MessageQ · 링버퍼 · 공유메모리

같은 PC 안에서 데이터를 옮기는 세 가지 도구

2-3

뮤텍스 · 세마포어

공유 자원을 안전하게 만들어 주는 동기화 수단

발표 대본

섹션 2 · 약 20 초

두 번째 주제입니다 . IPC 수단들의 개념과 동기화를 보겠습니다 .

세 부분으로 나눠서 , 2-1 에서 네트워크를 건너는 UDP 와 TCP/IP 를 , 2-2 에서 같은 PC 안에서 데이터를 옮기는 메시지 큐 · 링버퍼 · 공유메모리를 , 2-3 에서 이들을 안전하게 만들어 주는 뮤텍스와 세마포어를 보겠습니다 .

각각 특징 , 어떤 상황에서 쓰는지 , 장단점 순서로 정리했습니다 .

2-1 UDP · TCP / IP

같은 IP 위에 올라가지만, 신뢰성을 커널이 책임지느냐 애플리케이션이 책임지느냐가 다르다

TCP / IP

Transmission Control Protocol

연결형 · 바이트 스트림 · 신뢰성 보장

특징

- 3-way handshake 로 연결을 맺고 시작한다
- 순서 보장 · 유실 시 자동 재전송 · 중복 제거
- 흐름 제어와 혼잡 제어를 커널이 해준다
- 1 : 1 통신만 가능 (브로드캐스트 불가)

어떤 상황에서 쓰는가

- 한 건이라도 빠지면 안 되는 데이터
- 파일 전송, 명령 / 제어, 로그 수집
- 본 과제 ToDo#3 이 여기에 해당

장점

도착 · 순서 · 무결성을 커널이 보장
애플리케이션 코드가 단순해진다

단점

헤더 20 byte + 연결 유지 비용
재전송 때문에 지연이 될 수 있다

UDP

User Datagram Protocol

비연결형 · 데이터그램 · 최선 노력 전달

특징

- 연결 절차 없이 주소만 알면 바로 보낸다
- 보낸 크기 그대로 도착한다 (경계 보존)
- 도착 · 순서 · 중복을 보장하지 않는다
- 1 : N 브로드캐스트 · 멀티캐스트 가능

어떤 상황에서 쓰는가

- 최신 값만 의미 있는 주기 데이터
- 센서 · 텔레메트리, 음성 / 영상 스트리밍
- 여러 노드에 상태를 뿌릴 때

장점

헤더 8 byte, 연결 상태가 없어 가볍다
지연이 낮고 여러 상대에게 보낼 수 있다

단점

유실 · 순서 뒤바뀜을 앱이 처리해야
한 번에 보낼 수 있는 크기 제한 (64KB)

과제 적용

ToDo#3 은 TCP 를 사용한다. TCP 는 스트림이라 경계가 없다 — 4 byte 를 요청해도 2 byte 만 올라올 수 있으므로, 요청한 크기를 다 채울 때까지 반복 수신해야 한다 (`f_SocketRecvTCP_IPv4`)

발표 대본

2-1 UDP · TCP/IP · 약 2 분

둘 다 IP 위에 올라가지만, 신뢰성을 커널이 책임지느냐 애플리케이션이 책임지느냐가 다릅니다.

왼쪽 TCP 는 연결형입니다. 3-way handshake 로 연결을 맺고 시작하고, 순서 보장 · 유실 시 재전송 · 중복 제거를 커널이 다 해 줍니다. 그래서 한 건이라도 빠지면 안 되는 데이터 — 파일 전송, 명령 제어, 로그 수집에 씁니다. 오늘 ToDo#3 이 여기에 해당합니다. 단점은 헤더가 20 바이트에 연결 유지 비용이 있고, 재전송 때문에 지연이 될 수 있다는 점입니다.

오른쪽 UDP 는 비연결형입니다. 주소만 알면 바로 보내고, 보낸 크기 그대로 도착하는 대신 도착 자체를 보장하지 않습니다. 대신 헤더가 8 바이트로 가볍고 지연이 낮아서, 센서 데이터나 영상 · 음성 스트리밍처럼 " 최신 값만 의미 있는 " 주기 데이터에 씁니다. 브로드캐스트로 한 번에 여러 상대에게 뿌릴 수 있는 것도 UDP 만의 장점입니다.

아래가 실제 구현할 때 가장 중요한 포인트입니다. TCP 는 바이트 스트림이라 메시지 경계가 없습니다. 4 바이트를 요청해도 커널이 2 바이트만 줄 수 있습니다. 그래서 요청한 크기를 다 채울 때까지 반복해서 받아야 하고, 저희 코드의 `f_SocketRecvTCP_IPv4` 도 그렇게 구현돼 있습니다.

2-2 메시지 큐 Message Queue

커널이 관리하는 「메시지 단위」의 큐 — 보내는 쪽과 받는 쪽이 동시에 살아 있지 않아도 된다

메시지 큐 — 커널이 중간에서 보관한다



★ Windows 에는 System V 메시지 큐가 없다
→ ToDo#2 에서 공유메모리 + 링버퍼로 직접 구현했다

LINUX API

```
msgget() msgsnd() msgrcv() msgctl(IPC_RMID)
POSIX : mq_open / mq_send / mq_receive ( 우선순위 지원 )
```

특징

- 커널 공간에 큐가 만들어지고, 프로세스들은 키 (key) 로 같은 큐를 찾아 붙는다
- 바이트가 아니라 메시지 단위 — 보낸 크기 그대로 받는다 (경계 보존)
- mtype(메시지 타입)으로 원하는 종류만 골라 받을 수 있다
- 비면 받는 쪽이, 가득 차면 보내는 쪽이 커널에서 대기한다 (동기화 내장)

어떤 상황에서 쓰는가

- 명령 · 이벤트 · 상태 보고처럼 「건」 단위로 오가는 데이터
- 송신과 수신 속도가 다를 때, 그 차이를 큐가 흡수해 준다
- 두 프로세스의 기동 순서를 맞추기 어려울 때

장점

- 동기화가 커널에 내장 — 별도 락이 필요 없다
- 메시지 경계와 순서가 보장된다
- 비동기 — 상대가 아직 없어도 넣어둘 수 있다
- 타입별 선택 수신이 가능하다

단점

- 유저 ↔ 커널 복사 2 회 — 대용량엔 불리
- 큐 용량 제한 (기본 msgmnb 16KB)
- 지우지 않으면 커널에 계속 남는다
- 같은 PC 안에서만 동작한다

발표 대본

2-2 ① 메시지 큐 · 약 2 분

메시지 큐는 커널이 관리하는 "메시지 단위"의 큐입니다.

그림처럼 큐 자체가 커널 공간에 만들어지고, 프로세스들은 키를 가지고 같은 큐를 찾아 붙습니다. 프로세스 A가 msgsnd로 넣고, 프로세스 B가 msgrcv로 꺼냅니다.

특징은 네 가지입니다. 바이트가 아니라 메시지 단위라서 보낸 크기 그대로 받습니다. mtype이라는 필드로 원하는 종류만 골라 받을 수도 있습니다. 그리고 큐가 비면 받는 쪽이, 가득 차면 보내는 쪽이 커널에서 알아서 대기합니다. 동기화가 이미 들어 있다는 뜻입니다.

그래서 명령이나 이벤트처럼 "건" 단위로 오가는 데이터, 송수신 속도가 달라 완충이 필요한 경우, 그리고 두 프로세스의 기동 순서를 맞추기 어려울 때 씁니다. 커널에 남아 있으니 상대가 아직 안 떠 있어도 미리 넣어둘 수 있습니다.

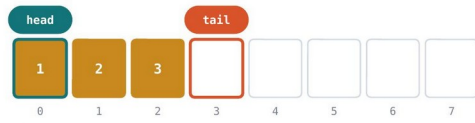
단점은 유저와 커널 사이 복사가 두 번 일어나서 대용량에 불리하고, 큐 용량이 기본 16KB 정도로 제한된다는 점입니다.

그리고 아래 표시한 것이 이번 과제의 핵심입니다. Windows에는 System V 메시지 큐가 아예 없습니다. 그래서 ToDo#2에서는 공유메모리와 링버퍼로 이 동작을 직접 만들었습니다. 그 이야기를 하려고 다음 두 장에서 링버퍼와 공유메모리를 먼저 보겠습니다.

2-2 링버퍼 Ring Buffer

고정 크기 배열을 원형으로 돌려 쓰는 자료구조 — 그 자체가 IPC 수단은 아니고, 데이터를 담는 「그릇」이다

고정 크기 배열 8 칸을 원형처럼 돌려 쓴다



끝에 닿으면 $(tail + 1) \% N$ 으로 0 번 슬롯으로 되돌아간다

head — 읽는 위치
소비자가 여기서 꺼낸다

tail — 쓰는 위치
생산자가 여기에 넣는다

```
ring[tail] = msg; tail = (tail+1) % N; count++;
```

본 과제 적용

쓰레드 간 8 슬롯 전역 링버퍼 (lpcThread.c)

프로세스 간 64 슬롯 링버퍼를 공유메모리 위에 (lpcMsgQ.c)

특징

- 고정 크기 배열 + 읽는 위치 (head) / 쓰는 위치 (tail) 두 개의 인덱스
- 끝에 닿으면 나머지 연산으로 처음으로 되돌아간다 → $tail = (tail + 1) \% N$
- 넣고 빼는 데 $O(1)$, 실행 중 동적 할당이 전혀 없다
- 「빔」과 「참」을 구별하려면 개수를 따로 세거나 한 칸을 비워 둔다

어떤 상황에서 쓰는가

- 생산자와 소비자의 속도가 다를 때 완충 장치로
- 시리얼 · 네트워크 수신 버퍼, 오디오 · 센서 스트림, 로그 버퍼
- 할당 지연이 허용되지 않는 실시간 코드

장점

- 넣기 / 빼기 모두 $O(1)$ 로 일정하다
- 메모리 고정 — 단편화도 할당 지연도 없다
- 연속 메모리라 캐시에 유리하다
- 공유메모리 위에 그대로 올릴 수 있다

단점

- 크기 고정 — 가득 차면 대기하거나 버려야
- head · tail 갱신 자체가 임계구역이다
- 동기화를 직접 붙여야 한다
- 가변 길이 데이터는 담기 번거롭다

발표 대본

2-2 ② 링버퍼 · 약 1 분 30 초

링버퍼는 앞의 두 개와 성격이 조금 다릅니다. 그 자체가 IPC 수단이 아니라, 데이터를 담는 자료구조입니다.

구조는 단순합니다. 고정 크기 배열에 읽는 위치 head 와 쓰는 위치 tail 두 개의 인덱스를 둡니다. 끝에 닿으면 나머지 연산으로 처음으로 돌아갑니다. 아래 코드처럼 tail 을 하나 늘리고 N 으로 나눈 나머지를 취하는 게 전부입니다.

넣고 빼는 데 $O(1)$ 이고 실행 중에 동적 할당이 전혀 없습니다. 그래서 생산자와 소비자의 속도가 다를 때 완충 장치로 쓰고, 시리얼이나 네트워크 수신 버퍼, 오디오 · 센서 스트림, 그리고 할당 지연이 허용되지 않는 실시간 코드에 씁니다.

단점은 크기가 고정이라 가득 차면 대기하거나 버려야 하고, head 와 tail 을 갱신하는 것 자체가 임계구역이라 동기화를 직접 붙여야 한다는 점입니다.

이번 과제에서는 쓰레드 간에는 8 슬롯 전역 링버퍼를, 프로세스 간에는 64 슬롯 링버퍼를 공유메모리 위에 올려서 썼습니다.

2-2 공유메모리 Shared Memory

서로 다른 프로세스의 가상 주소공간에 같은 물리 메모리를 매핑한다 — 붙고 나면 그냥 포인터 접근이다

같은 물리 메모리를 두 주소공간에 매핑



매핑 주소는 서로 다르다

물리 메모리 (같은 페이지)

데이터를 그대로 읽고 쓴다 · 복사 0 회
※ 순서를 지켜 주는 장치는 여기에 없다

공유메모리 + 뮤텝스 / 세마포어는 항상 한 세트다

API

Linux : shmget / shmat / shmdt / shmctl, shm_open + mmap
Windows : CreateFileMapping + MapViewOfFile

특징

- 커널은 「연결」만 해주고, 이후 읽기 · 쓰기에는 시스템 콜이 개입하지 않는다
- 복사가 0 회 — 모든 IPC 수단 중 가장 빠르다
- 프로세스마다 매핑되는 가상 주소가 다르다 → 포인터 대신 오프셋 · 인덱스를 쓴다
- 양쪽이 구조체 정렬 · 패딩을 똑같이 맞춰야 한다 (#pragma pack)

어떤 상황에서 쓰는가

- 영상 프레임 · 대형 버퍼처럼 크고 자주 오가는 데이터
- 지연이 중요한 실시간 처리
- 여러 프로세스가 같은 데이터를 동시에 읽어야 할 때

장점

- 가장 빠르다 — 복사 0 회, 시스템 콜 없음
- 크기 제한이 사실상 없다
- 자료구조를 그대로 올릴 수 있다
- 한 번 붙으면 접근 비용이 일정하다

단점

- 동기화 수단이 전혀 없다 → 반드시 직접 붙여야
- 한쪽 실수가 상대 메모리까지 깨뜨린다
- 포인터를 저장하면 안 된다 (주소가 다름)
- 누가 살아있는지 물라 수명 관리가 까다롭다

발표 대본

2-2 ③ 공유메모리 · 약 1 분 30 초

공유메모리는 서로 다른 프로세스의 가상 주소공간에 같은 물리 메모리를 매핑하는 방식입니다.

그림처럼 프로세스 A 와 B 가 각각 다른 가상 주소로 접근하지만, 실제로는 같은 물리 페이지를 보고 있습니다. 커널은 이 "연결"만 해 주고, 그 뒤 읽고 쓰기에는 시스템 콜이 전혀 개입하지 않습니다. 복사가 0 회라 IPC 중에서 가장 빠릅니다.

그래서 영상 프레임이나 대형 버퍼처럼 크고 자주 오가는 데이터, 지연이 중요한 실시간 처리에 씁니다.

주의할 점이 두 가지 있습니다. 첫째, 매핑되는 가상 주소가 프로세스마다 다르기 때문에 포인터를 그대로 저장하면 안 됩니다. 반드시 오프셋이나 인덱스를 써야 합니다. 저희 구현에서 head 와 tail 을 인덱스로 둔 이유가 그것입니다. 둘째, 양쪽이 구조체 정렬과 패딩을 똑같이 맞춰야 해서 #pragma pack 으로 고정했습니다.

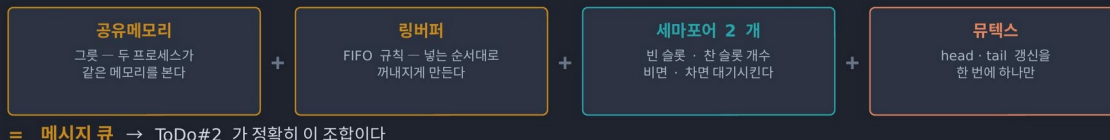
그리고 가장 중요한 단점은, 공유메모리에는 순서를 지켜 주는 장치가 전혀 없다는 것입니다. 그래서 공유메모리와 뮤텝스 · 세마포어는 항상 한 세트로 씁니다.

2-2 세 가지 비교 — 그리고 조합

메시지 큐는 「기능」, 링버퍼는 「자료구조」, 공유메모리는 「그릇」이다

구분	메시지 큐	링버퍼	공유메모리
정체	커널이 관리하는 IPC 객체	자료구조 (IPC 수단 아님)	커널이 연결해 주는 메모리
데이터 단위	메시지 — 경계 보존	슬롯 1 칸	바이트 — 경계 없음
복사 횟수	2 회 (유저 ↔ 커널)	1 회 (슬롯 write / read)	0 회
동기화	커널이 내장	직접 구현	직접 구현 (필수)
속도	보통	빠름	가장 빠름
용량	커널 한도 (기본 16KB)	고정 (컴파일 시 결정)	제한 거의 없음
대표 용도	명령 · 이벤트 전달	스트림 완충	대용량 데이터 공유

셋을 합치면 메시지 큐가 된다



발표 대본

2-2 ④ 비교와 조합 · 약 1 분 30 초

셋을 한 표로 정리하면 이렇습니다.

메시지 큐는 커널이 관리하는 IPC 객체이고, 링버퍼는 자료구조이고, 공유메모리는 커널이 연결해 주는 메모리입니다. 성격이 애초에 다릅니다.

복사 횟수를 보시면 메시지 큐가 유저와 커널 사이를 두 번 오가고, 공유메모리는 0 회입니다. 그래서 속도는 공유메모리가 가장 빠릅니다. 대신 동기화는 메시지 큐만 커널이 해 주고, 나머지 둘은 직접 구현해야 합니다.

그런데 아래를 보시면, 이 세 가지는 대체 관계가 아니라 조합할 수 있는 관계입니다. 공유메모리로 두 프로세스가 같은 메모리를 보게 하고, 그 위에 링버퍼를 올려서 넣은 순서대로 꺼내지게 만들고, 세마포어 두 개로 비었을 때와 찼을 때 대기시키고, 뮤텍스로 head 와 tail 을 보호하면 — 이것이 곧 메시지 큐입니다.

ToDo#2 가 정확히 이 조합입니다. Windows 에 메시지 큐가 없어서 이렇게 만들었고, 뒤에서 코드로 보여드리겠습니다.

2-3 뮤텝스 Mutex

상호배제 (MUTual EXclusion) — 임계구역에 한 번에 하나만 들어가게 하는 자물쇠

임계구역 — 한 번에 한 명만



임계구역 Critical Section

공유 데이터를 고치는 코드 — 동시에 들어오면 깨진다

MUTEX 잠금

기본 형태

lock(m); /* 공유 데이터 갱신 */ unlock(m);

API

Linux : pthread_mutex_lock / pthread_mutex_unlock
Windows : CRITICAL_SECTION · Named Mutex

특징

- 상태는 「잠금 / 풀림」 두 가지뿐이다
- 소유권이 있다 — 잠근 쓰레드만 풀 수 있다
- lock 에서 못 들어가면 커널이 재워 두었다가 unlock 때 깨운다
- 프로세스 내부는 CRITICAL_SECTION, 프로세스 간은 이름 있는 뮤텝스를 쓴다

어떤 상황에서 쓰는가

- 공유 변수 · 자료구조를 읽고 고칠 때
- 링버퍼의 head · tail 인덱스 갱신, 링크드 리스트 · 해시 테이블
- 파일 핸들 · 로그 버퍼처럼 한 번에 하나만 써야 하는 자원

장점

- 의미가 단순해 실수가 적고 디버깅이 쉽다
- 소유자가 분명 → 데드락 추적이 가능
- 우선순위 역전을 막는 기법을 쓸 수 있다
- 짧게 잡았다 푸는 데 오버헤드가 작다

단점

- 소유자만 풀 수 있어 「신호 전달」엔 못 쓴다
- 락 잡는 순서가 엇갈리면 데드락
- 임계구역이 길면 병렬성이 사라진다
- 조건을 기다리는 용도로 쓰면 바쁜 대기가 된다

발표 대본

2-3 ① 뮤텝스 · 약 1 분 30 초

이제 동기화입니다. 먼저 뮤텝스입니다.

이름 그대로 상호배제, MUTual EXclusion 입니다. 임계구역에 한 번에 하나만 들어가게 하는 자물쇠입니다. 그림처럼 쓰레드 1 이 lock 에 성공해서 들어가면, 쓰레드 2 는 unlock 될 때까지 블록됩니다.

핵심 특징은 소유권이 있다는 것입니다. 잠근 쓰레드만 풀 수 있습니다. 이 성질 덕분에 데드락이 났을 때 누가 잡고 있는지 추적할 수 있고, 우선순위 상속 같은 보호 기법도 쓸 수 있습니다.

쓰는 상황은 공유 변수나 자료구조를 읽고 고칠 때입니다. 링버퍼의 head·tail 갱신, 링크드 리스트, 파일 핸들처럼 한 번에 하나만 써야 하는 자원입니다.

단점을 하나만 강조하면, 소유자만 풀 수 있기 때문에 "신호 전달"에는 쓸 수 없다는 점입니다. "버퍼에 데이터가 들어올 때까지 기다린다" 같은 것을 뮤텝스만으로 하려면 락을 잡았다 풀었다 반복하는 바쁜 대기가 됩니다. 그래서 세마포어가 필요합니다.

2-3 세마포어 Semaphore

사용할 수 있는 자원의 「개수」를 세는 카운터 — wait 로 하나 가져가고 post 로 하나 돌려준다

카운터가 자원의 개수를 센다

count = 3

wait (P)

하나 가져간다
count = 0 이면 대기

post (V)

하나 돌려준다
대기 중인 쪽을 깨움

자원 슬롯

사용 가능

사용 가능

사용 가능

사용 중

사용 중

```
sem_wait(&sem); /* 자원 사용 */ sem_post(&sem);
```

API

```
Linux : sem_init / sem_wait / sem_post (POSIX)
       semget / semop (System V)
Windows : CreateSemaphore / WaitForSingleObject /
          ReleaseSemaphore
```

특징

- 0 이상의 정수 카운터. wait(P) : 1 감소, 0 이면 대기 / post(V) : 1 증가, 대기자를 깨움
- 소유권이 없다 → A 가 내리고 B 가 올려도 된다 (그래서 신호 전달이 된다)
- 카운팅 세마포어 (자원 N 개) 와 바이너리 세마포어 (0 · 1) 가 있다
- 대기는 커널이 재워 준다 → 바쁜 대기 (busy wait) 가 생기지 않는다

어떤 상황에서 쓰는가

- 개수가 정해진 자원 관리 — 커넥션 풀, 버퍼 슬롯
- 생산자-소비자에서 「찻다 / 비었다」를 알리는 신호
- 쓰레드 · 프로세스 간 실행 순서 맞추기

장점

- 개수를 셀 수 있다 (무텍스는 불가)
- 커널이 깨워 주므로 CPU 낭비가 없다
- 소유권이 없어 다른 주체가 신호를 줄 수 있다
- 이름을 주면 프로세스 간에도 쓸 수 있다

단점

- 소유자가 없어 잘못 쓴 지점을 찾기 어렵다
- post 를 한 번 빠뜨리면 영원히 대기
- 상호배제 용도로는 무텍스보다 안전하지 않다
- wait 순서를 잘못 짜면 데드락

발표 대본

2-3 ② 세마포어 · 약 1분 30초

세마포어는 사용할 수 있는 자원의 개수를 세는 카운터입니다.

wait, 교과서에서 P 연산이라고 하는 것은 카운터를 1 줄이고, 0 이면 대기합니다. post, V 연산은 카운터를 1 늘리고 대기 중인 쪽을 깨웁니다. 그림처럼 자원 슬롯이 5 개 중 3 개 남아 있으면 카운터가 3 입니다.

무텍스와 가장 큰 차이는 소유권이 없다는 것입니다. A 가 내리고 B 가 올려도 됩니다. 그래서 신호 전달이 가능합니다.

쓰는 상황은 세 가지입니다. 커넥션 풀이나 버퍼 슬롯처럼 개수가 정해진 자원 관리, 생산자-소비자에서 " 찻다 / 비었다 " 를 알리는 신호, 그리고 쓰레드나 프로세스 간 실행 순서를 맞추는 때입니다.

가장 큰 장점은 대기 중인 쪽을 커널이 깨워 준다는 것입니다. 바쁜 대기가 없어서 CPU 를 낭비하지 않습니다. 그리고 이름을 주면 프로세스 간에도 쓸 수 있는데, ToDo#2 가 그렇게 구현돼 있습니다.

단점은 소유자가 없어서 잘못 쓴 지점을 찾기 어렵고, post 를 한 번 빠뜨리면 영원히 대기한다는 점입니다.

2-3 뮤텍스 vs 세마포어 — 왜 둘 다 쓰는가

상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어. 하나로 다른 하나를 흉내내면 반드시 문제가 생긴다

구분	뮤텍스	세마포어
값	잠김 / 풀림 (0 · 1)	0 이상의 정수 카운터
소유권	있다 — 잠금 풀면 해제	없다 — 누구나 post 가능
목적	상호배제 (데이터 보호)	신호 · 개수 관리 (흐름 제어)
던지는 질문	"지금 나만 만지고 있나?"	"쓸 수 있는 게 몇 개 남았나?"
대표 용도	임계구역 보호	생산자-소비자, 자원 풀

뮤텍스만 쓰면

✗ "버퍼가 빌 때까지" 락을 잡았다 풀었다 반복 → 바쁜 대기 (busy wait) 로 CPU 를 태운다

세마포어만 쓰면

✗ 개수는 맞지만 head · tail 을 두 스레드가 동시에 고쳐 데이터가 깨진다

둘을 같이 쓰면

✓ 세마포어로 기다리고, 뮤텍스로 보호한다 → 바쁜 대기도 없고 데이터도 안전하다

생산자 - 소비자 표준 순서

송신 (생산자)

sem_wait(empty)

lock(mutex)

버퍼에 write

unlock(mutex)

sem_post(full)

수신 (소비자)

sem_wait(full)

lock(mutex)

버퍼에서 read

unlock(mutex)

sem_post(empty)

순서를 뒤집으면 데드락 — 뮤텍스를 먼저 잡고 세마포어를 기다리면, 상대는 그 락을 영원히 잡지 못한다.

발표 대본

2-3 ③ 뮤텍스 vs 세마포어 · 약 2 분

둘을 비교하면 핵심은 표 가운데 두 줄입니다. 소유권과 목적입니다.

뮤텍스는 소유권이 있고 목적이 상호배제, 즉 데이터 보호입니다. 던지는 질문은 "지금 나만 이 데이터를 만지고 있나?" 입니다.

세마포어는 소유권이 없고 목적이 신호와 개수 관리입니다. 질문은 "쓸 수 있는 게 몇 개 남았나?" 입니다.

오른쪽이 왜 둘 다 필요한지에 대한 답입니다.

뮤텍스만 쓰면, "버퍼가 빌 때까지 기다린다" 를 락을 잡았다 풀었다 반복하는 방식으로밖에 못 합니다. 바쁜 대기가 되어 CPU 를 태웁니다.

세마포어만 쓰면, 개수는 맞지만 두 스레드가 head 와 tail 을 동시에 고쳐서 데이터가 깨집니다.

그래서 둘을 같이 씁니다. 세마포어로 기다리고, 뮤텍스로 보호합니다.

아래가 생산자-소비자의 표준 순서입니다. 송신은 빈 슬롯 세마포어를 기다리고, 뮤텍스를 잡고, 쓰고, 풀고, 찬 슬롯 세마포어를 올립니다. 수신은 정확히 반대입니다.

마지막 줄이 중요합니다. 이 순서를 뒤집으면 데드락이 납니다. 뮤텍스를 먼저 잡은 상태에서 세마포어를 기다리면, 상대는 그 락을 영원히 잡지 못하고 세마포어를 올려 줄 수 없습니다. 그래서 반드시 세마포어가 먼저, 뮤텍스가 나중입니다.

03

데이터 송 / 수신 기능 구현 및 코드 정리

개발 환경

Windows · Visual Studio 2022 (v143) · x64
개발언어 C — IPC 코어는 전부 C DLL, 확인용 UI 만 MFC

공통 데이터 시나리오 (세 ToDo 모두 동일)

송신측 1 로 초기화된 int 를 0.1 초 간격으로 송신 , 이후 1 씩 증가 (100 까지)
수신측 받은 데이터를 그대로 출력

ToDo#1

쓰레드 간 메시지 송 / 수신

전역변수 + 뮤텍스 / 세마포어
IpcThread.c

ToDo#2

프로세스 간 메시지 송 / 수신

메시지 큐 (공유메모리 + 링버퍼)
IpcMsgQ.c

ToDo#3

다른 프로그램과 메시지 송 / 수신

TCP / IP (SocketUtility 포팅)
IpcSocket.c

IPC 로직은 전부 IpcCore.dll 안에 있고 , MFC UI 는 버튼으로 코어 함수를 호출하고 로그만 표시한다 .

발표 대본

섹션 3 · 약 1 분

이제 구현입니다 .

개발 환경은 과제 조건대로 Windows 에서 Visual Studio 2022, x64 구성이고 , 개발언어는 C 입니다 . IPC 기능은 전부 C 로 작성한 DLL 안에 있고 , MFC 는 버튼으로 코어 함수를 호출하고 로그만 표시하는 확인용 UI 입니다 . IPC 로직은 UI 에 한 줄도 두지 않았습니다 .

데이터 시나리오는 세 가지 ToDo 가 모두 동일합니다 . 송신측은 1 로 초기화된 int 를 0.1 초 간격으로 보내고 보낸 뒤 1 씩 증가시켜 100 까지 갑니다 . 수신측은 받은 데이터를 그대로 출력합니다 . 같은 시나리오를 쓰기 때문에 세 방식의 결과를 그대로 비교할 수 있습니다 .

ToDo#1 은 쓰레드 간 송수신을 전역변수와 뮤텍스 · 세마포어로 , ToDo#2 는 프로세스 간 송수신을 메시지 큐로 , ToDo#3 은 다른 프로그램과의 송수신을 TCP/IP 로 구현했고 각각 IpcThread.c, IpcMsgQ.c, IpcSocket.c 파일에 있습니다 .

ToDo#1 쓰레드 간 송 / 수신 구현

전역 링버퍼를 뮤텍스로 보호하고, 빈 슬롯 · 찬 슬롯 개수를 세마포어 2 개로 센다 · ipcThread.c



LINUX 대응

CRITICAL_SECTION → pthread_mutex_t
 CreateSemaphore / WaitForSingleObject → sem_init / sem_wait
 ReleaseSemaphore → sem_post
 _beginthreadex → pthread_create
 * CreateThread 대신 _beginthreadex — CRT 자원이 정리되도록

```
f_IpcThreadQueueSend()
// ① 빈 슬롯 대기 — 가득 차면 여기서 블록
if (WaitForSingleObject(s_hSemEmpty, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 뮤텍스로 링버퍼 상호배제
EnterCriticalSection(&s_stRingLock);
s_staRing[s_iRingTail] = *stpMsg;
s_iRingTail = (s_iRingTail + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount++;
LeaveCriticalSection(&s_stRingLock);

// ③ 찬 슬롯 +1 — 수신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemFull, 1, NULL);
```

```
f_IpcThreadQueueRecv()
// ① 찬 슬롯 대기 — 비어 있으면 여기서 블록
if (WaitForSingleObject(s_hSemFull, uiTimeOut_ms)
    != WAIT_OBJECT_0) return IPC_FAIL;

// ② 같은 뮤텍스로 상호배제
EnterCriticalSection(&s_stRingLock);
*stpMsg = s_staRing[s_iRingHead];
s_iRingHead = (s_iRingHead + 1) % IPC_THREAD_RING_SLOTS;
s_nRingCount--;
LeaveCriticalSection(&s_stRingLock);

// ③ 빈 슬롯 +1 — 송신 쓰레드를 깨운다
ReleaseSemaphore(s_hSemEmpty, 1, NULL);
```

발표 대본

ToDo#1 구현 · 약 2 분

ToDo#1 은 쓰레드 간 송수신입니다. 과제 조건이 전역변수에 뮤텍스와 세마포어를 쓰는 것이었습니다.

왼쪽 구조를 보시면, 송신 쓰레드와 수신 쓰레드 사이에 전역 링버퍼 8 슬롯을 두었습니다. 그리고 동기화 객체가 세 개입니다. 뮤텍스 역할의 CRITICAL_SECTION 하나로 head 와 tail 을 보호하고, 세마포어 두 개로 빈 슬롯과 찬 슬롯의 개수를 셉니다. 빈 슬롯 세마포어는 초기값 8, 찬 슬롯 세마포어는 초기값 0 으로 시작합니다.

오른쪽이 실제 코드입니다. 송신은 세 단계입니다. 먼저 빈 슬롯 세마포어를 기다립니다. 버퍼가 가득 차 있으면 여기서 블록됩니다. 그다음 뮤텍스를 잡고 링버퍼에 쓰고 tail 을 하나 돌립니다. 마지막으로 찬 슬롯 세마포어를 올려서 수신 쓰레드를 깨웁니다.

수신은 정확히 대칭입니다. 찬 슬롯을 기다리고, 같은 뮤텍스로 보호하고, 읽고, 빈 슬롯을 올립니다. 앞에서 본 생산자-소비자 표준 순서 그대로입니다.

왼쪽 아래는 리눅스 대응입니다. CRITICAL_SECTION 은 pthread_mutex_t, CreateSemaphore 와 WaitForSingleObject 는 sem_init 과 sem_wait, ReleaseSemaphore 는 sem_post, _beginthreadex 는 pthread_create 에 대응합니다. 참고로 쓰레드 생성에 CreateThread 가 아니라 _beginthreadex 를 쓴 이유는, CRT 를 쓰는 쓰레드는 이쪽으로 만들어야 종료할 때 CRT 자원이 정리되기 때문입니다.

ToDo#1 실행 결과

한 프로세스 안에서 [Start] — 송신과 수신이 0.1 초 간격으로 교대로 찍힌다

```

● IpcUI — 로그창 · Thread [Start] → [Stop]

14:20:31 [TH] start
14:20:31 [TH] tx 1
14:20:31 [TH] rx 1
14:20:31 [TH] tx 2
14:20:31 [TH] rx 2
14:20:31 [TH] tx 3
14:20:31 [TH] rx 3
14:20:32 [TH] tx 4
14:20:32 [TH] rx 4
14:20:32 [TH] tx 5
14:20:32 [TH] rx 5
...
14:20:41 [TH] tx 98
14:20:41 [TH] rx 98
14:20:41 [TH] tx 99
14:20:41 [TH] rx 99
14:20:41 [TH] tx 100
14:20:41 [TH] rx 100
14:20:41 [TH] tx done (100)
14:20:46 [TH] rx done (100)
14:20:46 [TH] stop
  
```

무엇을 확인했나

1 1 부터 100 까지 빠짐없이

송신 100 건, 수신 100 건. 순서가 뒤바뀌거나 빠진 값이 없다.

2 tx 와 rx 가 교대로

링버퍼가 8 칸뿐이라 수신이 밀리면 송신이 세마포어에서 대기한다 — 흐름 제어가 동작한다는 뜻이다.

3 대기 중에도 [Stop] 에 즉시 반응

수신 쓰레드는 100ms 단위로 타임아웃을 두고 대기하므로 정지 요청이 바로 먹힌다.

실행 방법

IpcUI.exe 를 실행하고 Thread 영역의 [Start] 버튼 하나만 누르면 된다.
한 프로세스 안에서 송신 쓰레드와 수신 쓰레드가 한 쌍으로 뜬다.

발표 대본

ToDo#1 실행 결과 · 약 1 분

실행 결과입니다. IpcUI 를 띄우고 Thread 영역의 [Start] 버튼 하나만 누르면 한 프로세스 안에서 송신 쓰레드와 수신 쓰레드가 한 쌍으로 뜬다.

로그를 보시면 start 이후 tx 1, rx 1, tx 2, rx 2 순서로 0.1 초 간격으로 찍히고, 100 까지 간 뒤 tx done (100) 이 나옵니다. 송신 100 건, 수신 100 건이 순서대로 빠짐없이 도착했습니다.

두 번째로 볼 것은 tx 와 rx 가 교대로 찍힌다는 점입니다. 링버퍼가 8 칸뿐이라 수신이 밀리면 송신이 세마포어에서 대기하게 됩니다. 흐름 제어가 실제로 동작하고 있다는 뜻입니다.

세 번째는 정지 반응입니다. 송신은 100 까지 보내고 스스로 끝나지만 수신 쓰레드는 계속 대기합니다. 그런데 수신은 100 밀리초 단위로 타임아웃을 두고 기다리기 때문에, [Stop] 을 누르면 대기 중이어도 바로 빠져나와 rx done 을 찍고 종료합니다.

ToDo#2

메시지 큐가 없다 — 무엇으로 대체했나

과제는 메시지 큐를 요구했지만 개발 환경이 Windows 다 · lpcMsgQ.c



문제

Windows 에는 **System V** 메시지 큐가 없다

msgget · msgsnd · msgrcv · msgctl 이 아예 존재하지 않는다



해법

공유메모리 + 링버퍼로 메시지 큐를 직접 만들었다

네임드 공유메모리 + 네임드 뮉텍스 1 개 + 네임드 세마포어 2 개

Linux (System V)	본 과제의 Windows 구현	무엇이 같은가
msgget(key, IPC_CREAT)	CreateFileMapping — 네임드 공유메모리 + 링버퍼 헤더 초기화	이름 하나로 같은 큐를 찾아 붙는다
msgsnd()	세마포어 (빈슬롯) 대기 → 뮉텍스 → 링버퍼 write → 세마포어 (찬슬롯) +1	가득 차면 보내는 쪽이 대기한다
msgrcv()	세마포어 (찬슬롯) 대기 → 뮉텍스 → 링버퍼 read → 세마포어 (빈슬롯) +1	비어 있으면 받는 쪽이 대기한다
msgctl(IPC_RMID)	모든 핸들 CloseHandle	마지막 참조가 사라지면 정리된다
mtype	ST_lpcMsg.iMsgType	메시지 종류를 구분하는 필드

공유메모리

두 프로세스가 같은 메모리를 본다

+ 링버퍼

넣은 순서대로 꺼내진다 (FIFO)

+ 세마포어 2 개

비어 · 차면 커널이 재워 준다

+ 뮉텍스

head · tail 을 한 번에 하나만

발표 대본

ToDo#2 구현 ① · 약 2 분

ToDo#2 는 프로세스 간 송수신이고, 과제는 메시지 큐로 구현하라고 되어 있었습니다. 그런데 개발 환경이 Windows 입니다.

문제는 명확합니다. Windows 에는 System V 메시지 큐가 없습니다. msgget, msgsnd, msgrcv, msgctl 이 아예 존재하지 않습니다. 리눅스 헤더를 포함할 수도 없고 대응되는 API 도 없습니다.

그래서 해법은, 앞에서 본 조합을 그대로 쓰는 것이었습니다. 네임드 공유메모리 위에 링버퍼를 올리고, 네임드 뮉텍스 하나와 네임드 세마포어 두 개를 붙여서 메시지 큐와 같은 동작을 직접 만들었습니다.

가운데 표가 대응 관계입니다.

msgget 은 CreateFileMapping 으로 네임드 공유메모리를 만들고 링버퍼 헤더를 초기화하는 것으로 대응합니다. 이름 하나로 같은 큐를 찾아 붙는다는 의미가 같습니다.

msgsnd 는 빈 슬롯 세마포어를 기다리고, 뮉텍스를 잡고, 링버퍼에 쓰고, 찬 슬롯 세마포어를 올리는 순서입니다. 가득 차면 보내는 쪽이 대기한다는 의미가 그대로 유지됩니다.

msgrcv 는 그 반대이고, 비어 있으면 받는 쪽이 대기합니다.

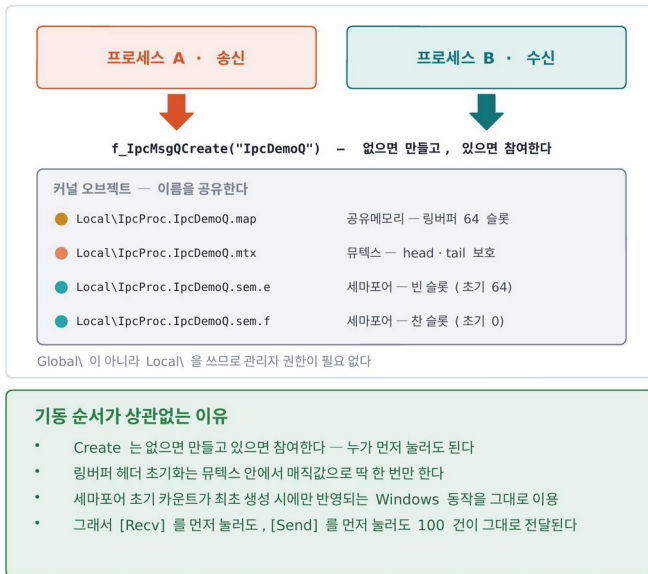
msgctl 의 IPC_RMID 는 모든 핸들을 CloseHandle 하는 것으로, mtype 은 구조체의 iMsgType 필드로 대응했습니다.

정리하면 아래 네 가지 조합입니다. 공유메모리가 그릇, 링버퍼가 FIFO 규칙, 세마포어 두 개가 대기 조건, 뮉텍스가 상호배제입니다. 이 네 개를 합쳐야 비로소 메시지 큐가 됩니다.

ToDo#2

커널 오브젝트 구성과 송 / 수신 절차

두 프로세스는 「같은 이름」만 알면 같은 큐에 붙는다 · 기동 순서는 상관없다



```
f_IpcMsgQSend()  ≡  msgsnd()
// ① 빈 슬롯 대기 (프로세스 간 세마포어)
WaitForSingleObject(stpQ->vpSemEmpty, uiTimeOut_ms);

// ② 프로세스 간 상호배제 — 네임드 뮤텝스
WaitForSingleObject(stpQ->vpMutex, 50000);

// ③ 공유메모리 위의 링버퍼에 write
stpRing->staSlot[stpRing->iTail] = *stpMsg;
stpRing->iTail = (stpRing->iTail + 1) % stpRing->iCapacity;
stpRing->nCount++;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemFull, 1, NULL); // ④ 찬 슬롯 +1
```

```
f_IpcMsgQRecv()  ≡  msgrcv()
// ① 찬 슬롯 대기
WaitForSingleObject(stpQ->vpSemFull, uiTimeOut_ms);

// ② 같은 네임드 뮤텝스
WaitForSingleObject(stpQ->vpMutex, 50000);

// ③ 링버퍼에서 read
*stpMsg = stpRing->staSlot[stpRing->iHead];
stpRing->iHead = (stpRing->iHead + 1) % stpRing->iCapacity;
stpRing->nCount--;

ReleaseMutex(stpQ->vpMutex);
ReleaseSemaphore(stpQ->vpSemEmpty, 1, NULL); // ④ 빈 슬롯 +1
```

발표 대본

ToDo#2 구현 ② · 약 2 분

실제 구성을 보겠습니다.

왼쪽입니다. 두 프로세스는 f_IpcMsgQCreate 에 같은 큐 이름 — 여기서는 IpcDemoQ — 만 넘기면 같은 큐에 붙습니다. 커널 오브젝트는 네 개입니다. .map 이 공유메모리로 64 슬롯 링버퍼가 올라가고, .mtx 가 뮤텝스, .sem.e 가 빈 슬롯 세마포어로 초기값 64, .sem.f 가 찬 슬롯 세마포어로 초기값 0 입니다.

이름 앞에 Global 이 아니라 Local 을 붙였습니다. Global 은 관리자 권한이 필요하지만 Local 은 같은 로그인 세션 안에서 동작해서 권한 없이 실행할 수 있습니다.

왼쪽 아래가 이 구현에서 신경 쓴 부분입니다. 어느 프로세스를 먼저 띄워도 동작해야 했습니다. 그래서 Create 함수를 "없으면 만들고 있으면 참여한다" 로 만들었고, 링버퍼 헤더 초기화는 뮤텝스를 잡은 안에서 매직값을 확인해 딱 한 번만 하도록 했습니다. 세마포어는 Windows 가 초기 카운트를 최초 생성 시에만 반영하기 때문에, 두 번째로 붙는 프로세스가 카운트를 덮어쓸 걱정이 없습니다.

오른쪽 코드가 실제 송수신입니다. 앞에서 본 순서와 완전히 같습니다. 세마포어 대기, 뮤텝스, 링버퍼 접근, 뮤텝스 해제, 반대쪽 세마포어 증가입니다. 다른 점은 쓰레드용이 아니라 프로세스 간 커널 오브젝트를 쓴다는 것뿐입니다. 결국 msgsnd 와 msgrcv 를 이 코드가 대신하고 있습니다.

ToDo#2 실행 결과

[New Process] 로 창을 하나 더 띄우고, 한쪽은 [Recv] · 다른쪽은 [Send] 를 누른다

● 프로세스 ① 수신 — [Recv] 를 먼저 눌러 큐를 만든다

```
14:22:05 [MQ] queue 'IpcDemoQ' open
14:22:05 [MQ] start (rx)
14:22:12 [MQ] rx 1
14:22:12 [MQ] rx 2
14:22:12 [MQ] rx 3
14:22:12 [MQ] rx 4
14:22:12 [MQ] rx 5
14:22:13 [MQ] rx 6
...
14:22:22 [MQ] rx 98
14:22:22 [MQ] rx 99
14:22:22 [MQ] rx 100
14:22:28 [MQ] rx done (100)
14:22:28 [MQ] stop
```

● 프로세스 ② 송신 — [New Process] 로 띄운 창에서 [Send]

```
14:22:12 [MQ] queue 'IpcDemoQ' open
14:22:12 [MQ] start (tx)
14:22:12 [MQ] tx 1
14:22:12 [MQ] tx 2
14:22:12 [MQ] tx 3
14:22:12 [MQ] tx 4
14:22:12 [MQ] tx 5
14:22:13 [MQ] tx 6
...
14:22:22 [MQ] tx 98
14:22:22 [MQ] tx 99
14:22:22 [MQ] tx 100
14:22:22 [MQ] tx done (100)
14:22:26 [MQ] stop
```

1

두 프로세스 사이로 100 건이 그대로 전달

송신 창의 tx 1~100 과 수신 창의 rx 1~100 이 정확히 일치한다.

2

수신을 먼저 눌러도, 송신을 먼저 눌러도 동작

Create 가 없으면 만들고 있으면 참여하므로 기동 순서에 의존하지 않는다.

3

수신 창의 시각이 7 초 늦게 시작

큐가 먼저 만들어져 대기하고 있다가, 송신이 붙는 순간부터 받기 시작한다 — 비동기 동작 확인.

발표 대본

ToDo#2 실행 결과 · 약 1 분 30 초

실행 결과입니다. 확인 방법은 [New Process] 버튼으로 같은 프로그램 창을 하나 더 띄우고, 한쪽에서 [Recv], 다른쪽에서 [Send] 를 누르는 것입니다.

왼쪽이 수신 프로세스입니다. 22 분 05 초에 queue 'IpcDemoQ' open 이 찍히고 rx 로 시작합니다. 오른쪽 송신 프로세스는 7 초 뒤인 22 분 12 초에 같은 큐에 붙어서 tx 1 부터 보내기 시작합니다.

여기서 세 가지를 확인할 수 있습니다.

첫째, 송신 창의 tx 1 부터 100 까지와 수신 창의 rx 1 부터 100 까지 정확히 일치합니다. 두 프로세스 사이로 100 건이 그대로 전달됐습니다.

둘째, 수신을 먼저 눌러도 송신을 먼저 눌러도 동작합니다. Create 가 없으면 만들고 있으면 참여하기 때문에 기동 순서에 의존하지 않습니다.

셋째, 시각을 보시면 수신 쪽이 7 초 먼저 떠서 큐를 만들어 놓고 대기하다가, 송신이 붙는 순간부터 받기 시작했습니다. 메시지 큐의 비동기 성질이 그대로 재현된 것입니다.

ToDo#3 TCP / IP 구현

첨부받은 Linux SocketUtility.h / .c v5.0 을 Winsock2 로 포팅 — 함수명 · 인자 · 상수는 원본 그대로 · lpcSocket.c

Tx · 송신 = 클라이언트

socket

connect

send

Rx · 수신 = 서버

socket

bind

listen

accept

recv

accept 는 200ms 단위 select 루프, connect 는 논블로킹 + select — 정지 요청에 바로 반응한다

포팅에서 실제로 걸린 것 — 컴파일은 통과하는데 값이 달라지는 항목

항목	Linux	Windows
소켓 핸들	int	SOCKET (UINT_PTR) · x64 에서 64bit
송 / 수신 타임아웃	timeval 구조체	DWORD 밀리초 — 그대로 넘기면 엉뚱한 값
소켓 닫기	close()	closesocket()
초기화	없음	WSAStartup / WSACleanup 필요
에러 확인	errno	WSAGetLastError()
select 1 번 인자	maxfd + 1	무시된다

※ 타임아웃은 형이 맞아 보여 그냥 지나치기 쉽다 — 내부에서 timeval → DWORD ms 로 변환했다

데모 송신 — 프레임 헤더 없이 int 4 byte

```
for (iData = IPC_DEMO_FIRST_VALUE; iData <= IPC_DEMO_LAST_VALUE; iData++) {
    // htonl 체크박스가 켜져 있으면 네트워크 바이트 순서로
    iWire = (s_iDemoUseNB0 != 0) ? htonl(iData) : iData;

    f_SocketSendTCP_IPv4(&s_stDemoSocket,
        &iWire, sizeof(iWire));

    // 0.1초 대기 — 정지 요청도 같이 확인한다
    f_IsDemoStopRequested(IPC_DEMO_INTERVAL_MS);
}
```

데이터를 어떻게 실어 보냈나

- int 4 byte 를 프레임 헤더 없이 그대로 보낸다
- x64 Windows 와 x86-64 Linux 는 둘 다 little-endian 이라 기본값으로 값이 맞는다
- 상대가 htonl 을 쓰면 UI 의 htonl 체크박스를 켜서 맞춘다

송 / 수신은 요청한 크기를 다 채울 때까지 반복한다 · 반환값 >0 처리 바이트 | 0 타임아웃 | -1 에러 · 상대 종료

발표 대본

ToDo#3 구현 · 약 2 분

ToDo#3 은 다른 사람 프로그램과 메시지를 주고받는 것이고, 첨부받은 Linux SocketUtility 코드를 참고해서 TCP/IP 로 구현했습니다. 원본이 리눅스용이라 Winsock2 로 포팅했는데, 상대 코드와 맞물려야 하니 함수명과 인자, 상수, enum 은 원본 그대로 두었습니다.

역할은 Tx 가 클라이언트로 connect 하고, Rx 가 서버로 bind, listen, accept 하는 구조입니다. UDP 쪽에서 Tx 는 목적지 주소를, Rx 는 자기 bind 주소를 받기 때문에 TCP 도 똑같이 맞췄습니다. 상대가 반대로 동작하면 Init 함수만 바꿔 부르면 됩니다.

정지 요청에 바로 반응해야 해서, accept 는 200 밀리초 단위 select 루프로 잘게 나눴고 connect 는 논블로킹에 select 를 붙였습니다. 블로킹 connect 는 실패가 확정되기까지 20 초 넘게 걸립니다.

가운데 표가 포팅에서 실제로 걸렸던 것들입니다. 소켓 핸들이 리눅스는 int 인데 Windows 는 SOCKET, x64 에서 64 비트입니다. close 는 closesocket, errno 는 WSAGetLastError 로 바뀌고 WSAStartup 도 필요합니다.

빨간색 표시한 타임아웃이 가장 위험했습니다. 리눅스는 setsockopt 에 timeval 구조체를 넘기는데 Windows 는 DWORD 밀리초를 받습니다. 타입 캐스팅이 되니까 컴파일도 통과하고 경고도 안 납니다. 그런데 값의 의미가 완전히 달라져서 런타임에만 이상하게 동작합니다. 그래서 내부에서 timeval 을 밀리초로 변환하도록 했습니다.

오른쪽 코드가 송신 루프입니다. int 4 바이트를 프레임 헤더 없이 그대로 보냅니다. x64 Windows 와 x86-64 Linux 는 둘 다 little-endian 이라 기본값으로 값이 맞고, 상대가 htonl 을 쓰면 UI 의 체크박스를 켜서 맞춥니다.

ToDo#3 실행 결과

한쪽은 [Recv](서버), 다른쪽은 [Send](클라이언트) — 다른 PC 의 프로그램과도 같은 방식으로 붙는다

● 프로세스 ① 수신 = 서버 — [Recv]	● 프로세스 ② 송신 = 클라이언트 — [Send]
<pre> 14:25:02 [TCP] start (rx) 14:25:02 [TCP] listen 0.0.0.0:51000 14:25:09 [TCP] accept 127.0.0.1:60312 14:25:09 [TCP] rx 1 14:25:09 [TCP] rx 2 14:25:09 [TCP] rx 3 14:25:10 [TCP] rx 4 14:25:10 [TCP] rx 5 ... 14:25:19 [TCP] rx 98 14:25:19 [TCP] rx 99 14:25:19 [TCP] rx 100 14:25:20 [TCP] peer closed 14:25:20 [TCP] rx done (100) </pre>	<pre> 14:25:09 [TCP] start (tx) 14:25:09 [TCP] connecting 127.0.0.1:51000 14:25:09 [TCP] connected 127.0.0.1:51000 14:25:09 [TCP] tx 1 14:25:09 [TCP] tx 2 14:25:09 [TCP] tx 3 14:25:10 [TCP] tx 4 14:25:10 [TCP] tx 5 ... 14:25:19 [TCP] tx 98 14:25:19 [TCP] tx 99 14:25:19 [TCP] tx 100 14:25:19 [TCP] tx done (100) 14:25:20 [TCP] stop </pre>

1 연결 → 100 건 → 정상 종료

accept 로 상대 IP · 포트를 확인하고, 100 건을 받은 뒤 상대가 끊으면 peer closed 로 마무리된다.

2 다른 PC 와 붙일 때

수신측 IP 를 0.0.0.0 으로 두고 방화벽에서 포트를 연다. 상대가 htonl 을 쓰면 체크박스를 켜다.

3 역할이 반대인 상대와도 가능

Tx / Rx Init 함수를 바꿔 부르지만 하면 된다. 함수명 · 상수는 원본 SocketUtility 와 동일하게 두었다.

발표 대본

ToDo#3 실행 결과 · 약 1 분 30 초

TCP 실행 결과입니다. 마찬가지로 창을 두 개 띄우고 한쪽은 [Recv] 로 서버, 다른쪽은 [Send] 로 클라이언트를 시작합니다.

왼쪽 서버 로그를 보시면 listen 0.0.0.0:51000 으로 대기하다가, 7 초 뒤 클라이언트가 붙으면서 accept 에 상대 IP 와 포트가 찍힙니다. 그리고 rx 1 부터 100 까지 받습니다. 클라이언트가 100 건을 다 보내고 소켓을 닫으면 peer closed 가 뜨고 rx done (100) 으로 마무리됩니다.

오른쪽 클라이언트는 connecting, connected 이후 tx 1 부터 100 까지 보내고 tx done (100) 으로 끝납니다. 양쪽 카운트가 100 으로 일치합니다.

다른 PC 와 붙일 때는 수신측 IP 를 0.0.0.0 으로 두고 방화벽에서 포트를 열면 됩니다. 상대가 htonl 을 쓰면 체크박스만 켜면 되고, 상대의 Tx/Rx 역할이 반대라면 Init 함수를 바꿔 부르면 됩니다. 함수명과 상수를 원본 SocketUtility 와 동일하게 유지한 것이 이런 상황에서 도움이 됩니다.

정리

개념 세 가지와, 그 개념이 코드에서 어떻게 만났는지

1

뮤텍스와 세마포어는 짝이다

상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어.
하나로 다른 하나를 흉내내면 바쁜 대기가 되거나 데이터가 깨진다.

2

메시지 큐는 없으면 만들 수 있다

공유메모리 (그릇) + 링버퍼 (FIFO) + 세마포어 2 개 + 뮤텍스.
Windows 에 System V 메시지 큐가 없어 이 조합으로 msgsnd / msgrcv 와 같은 의미를 만들었다.

3

포팅에서 위험한 것은 「타입이 맞는」 차이다

SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을
먼저 찾아야 한다. 소켓 핸들 크기, 타임아웃 단위가 대표적이다.

검증 결과 ToDo#1 · #2 · #3 모두 1 부터 100 까지 0.1 초 간격으로 송신하고, 수신측이 같은 값을 빠짐없이 출력하는 것을 확인했다.

발표 대본

정리 · 약 1 분

정리하겠습니다. 세 가지를 남기고 싶습니다.

첫째, 뮤텍스와 세마포어는 짝입니다. 상호배제는 뮤텍스, 개수를 세고 기다리는 것은 세마포어입니다. 하나로 다른 하나를 흉내내면 바쁜 대기가 되거나 데이터가 깨집니다. 셋 중 어느 ToDo 를 봐도 이 조합이 그대로 들어가 있습니다.

둘째, 메시지 큐는 없으면 만들 수 있습니다. 공유메모리라는 그릇, 링버퍼라는 FIFO 규칙, 세마포어 두 개, 뮤텍스 하나를 합치면 msgsnd 와 msgrcv 와 같은 의미가 됩니다. Windows 에 System V 메시지 큐가 없다는 제약이 오히려 메시지 큐가 내부적으로 무엇을 하고 있는지 이해하는 계기가 됐습니다.

셋째, 포팅에서 위험한 것은 타입이 맞아 보이는 차이입니다. SO_RCVTIMEO 처럼 컴파일은 통과하는데 값의 의미가 달라지는 항목을 먼저 찾아야 합니다.

검증 결과, ToDo#1, #2, #3 모두 1 부터 100 까지 0.1 초 간격으로 송신하고 수신측이 같은 값을 빠짐없이 출력하는 것을 확인했습니다.

이상 발표를 마치겠습니다. 감사합니다.